

Introduction

Transfer-Optimization (FIFO)



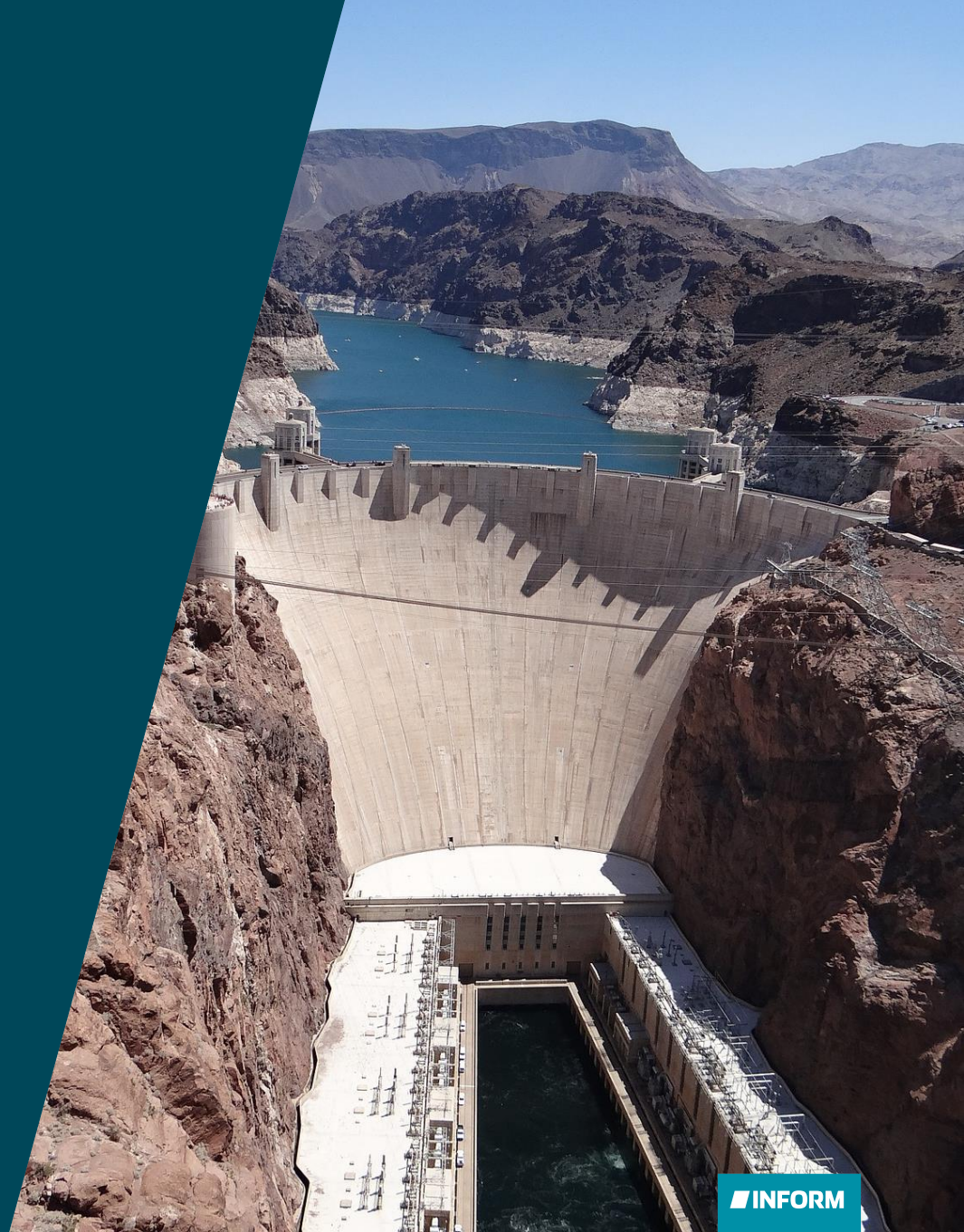
Privacy Notice: This document and its content is the absolute property of INFORM GmbH and/or its subcontractors. The reproduction, distribution and utilization of this document as well as the communication of its contents to others without express authorization is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design.

Agenda

- Introduction
- Constraints
- Transformations
- Optimization Approach
- Additional Parameters
- Conclusion



INTRODUCTION





Overview

Objective: Schedule all deliveries with as little delay as possible

Secondary goals:

- Avoid unnecessary trips
- Comply with First-in-First-out (*FiFo*)-rule where first vehicle gets next load

General: Minimize the number of open deliveries and the delay of deliveries.

Including various constraints such as special treatment for the early morning (*First Round*).



Approach

Heuristic approach with two-step procedure, since exact methods are too computationally intensive

1. Plant Dispatch

- Work on plant basis, i.e., vehicles on site are used to deliver orders on site (principal plant is used if available), and comply with FiFo-rule
- Delays are possible

2. Transformations

- Exchange work (deliveries) and/or resources (vehicles) between plants

Results:

Schedule with information for each delivery (loading times, plant, vehicle) → *Proposals*

CONSTRAINTS





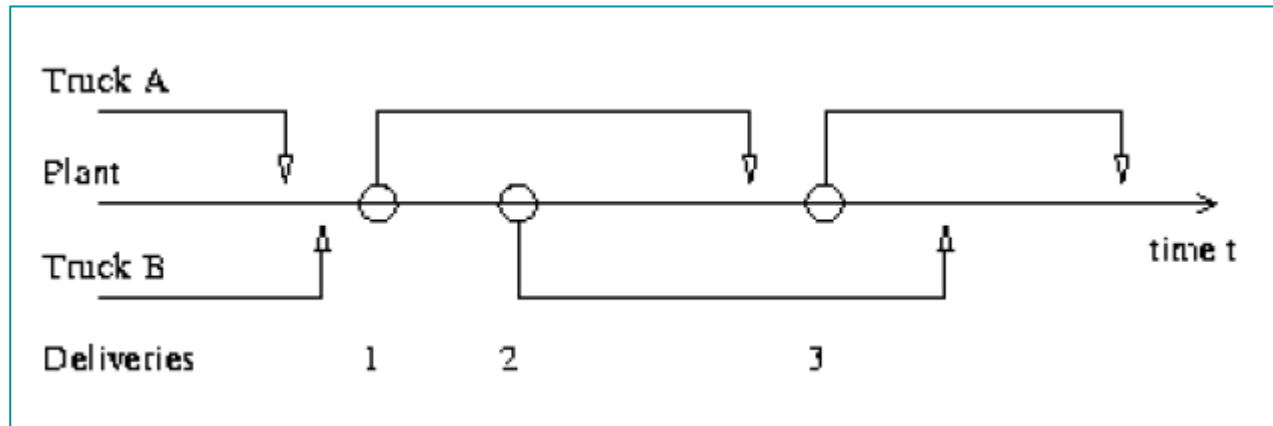
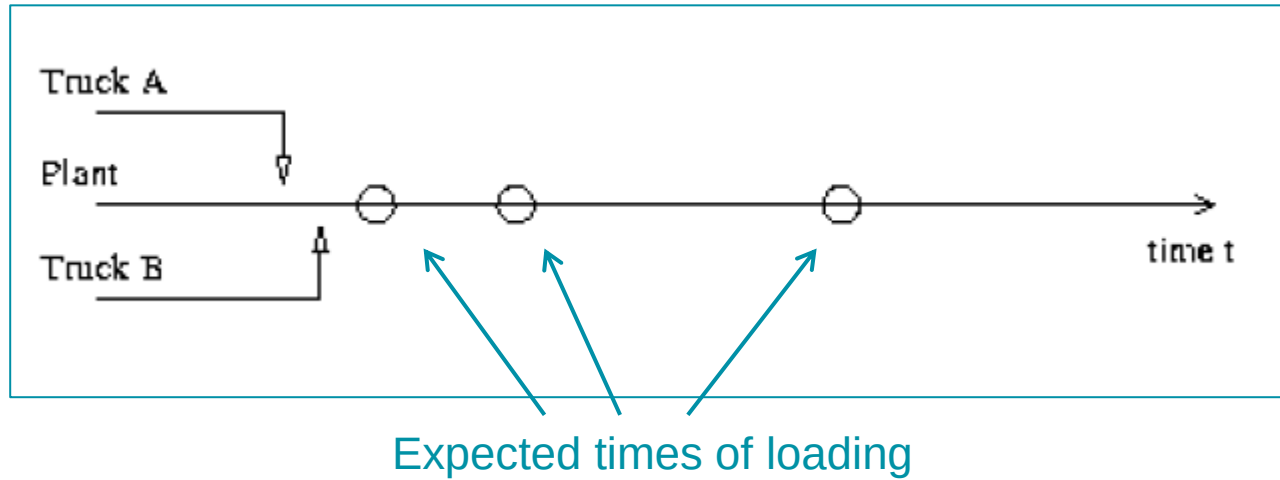
Input Data

- **Plants**
- **Vehicles/trucks**
- **Order**
 - Quantity (min, max)
 - Continuous delivery flow and *load-spacing*
 - Customer
- **(Assigned) deliveries with delivery time intervals**
- **Principal plant** (usually nearest plant) and **alternative plants**

Constraints

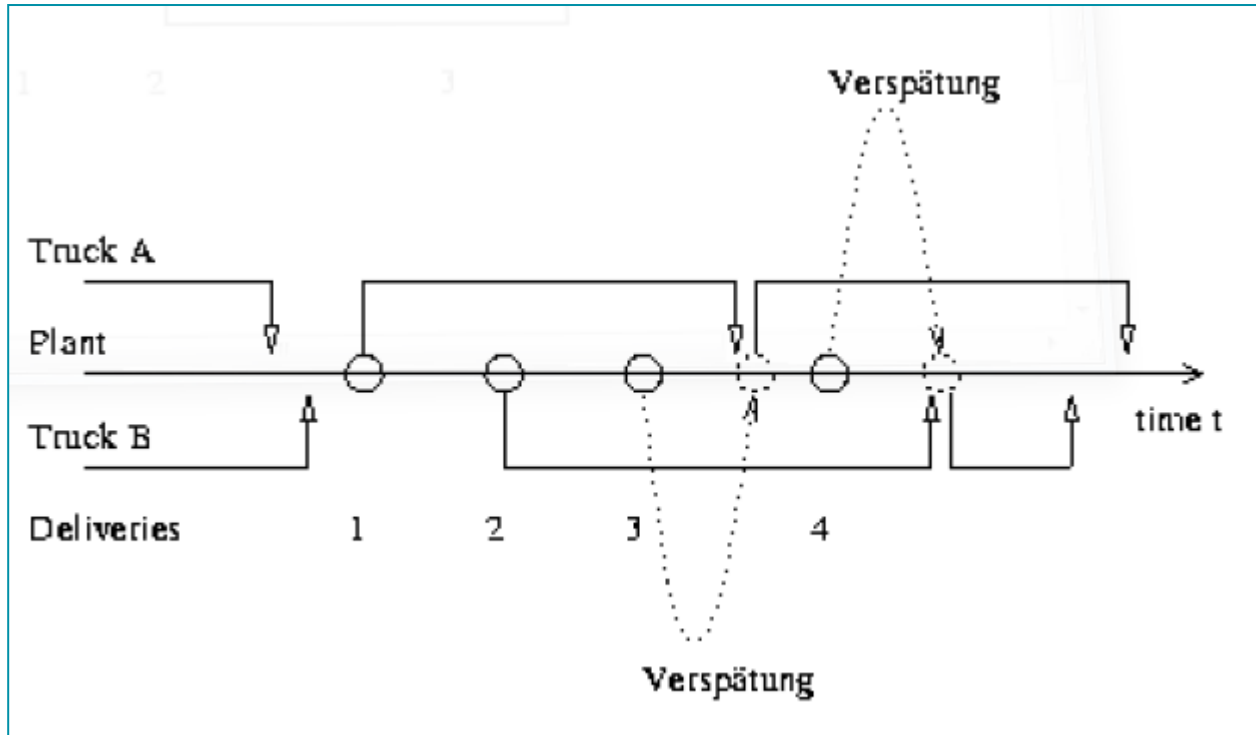
- Shift and working times for single shift functionality and planning for one day of a *Dispatch Group* (DG)
- FiFo-rule at concrete plants with exceptions
- First Round-rule: Deviation from planned start time in the morning is considered (fairness)
- Priorities and delivery time window of orders (defined via start of first delivery and delivery flow)
- Continuous delivery flow with specified quantity per time unit and predecessor orders with offset
- Delays of deliveries are allowed – non-linear cost function
- *Early loading* is allowed
- Properties/exclusions of vehicles/shifts, orders und plants
- Vehicle capacities and priorities and *Plusloads*
- Opening hours of plants, parking and customer locations
- Maximum capacity per concrete plant
- Number of loading slots at plants (*legs*) and alternative plants
- Cleaning durations and fixed and variable unloading durations

FiFo-Rule



- First vehicle at plant gets next delivery
- FiFo-rule very restrictive
→ Fast assignment of vehicles and deliveries
- Approach considers vehicles that arrived in the past at the plants → idle time
- .. and vehicles that arrive in the future
- Exceptions of FiFo-rule
 - Properties (vehicle and delivery do not match!)
 - First Round-rule
 - High-priority-orders
 - ...

FiFo-Rule

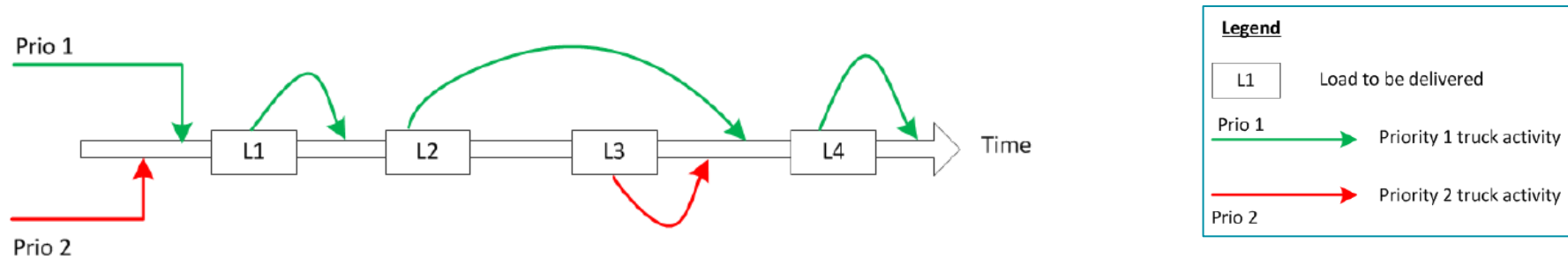


- Situation with delay
- Note: All following deliveries of the same order must be postponed → Load-spacing
- See GUI
→ Delivery Overview

First Round and Morning Roster

- Exception rule for first day's work assignment
- Prefer vehicles that have not worked before – DG-parameter `thePreferNotYetWorked`
- Dispatcher determines *Morning Roster* (MR) at plant that defines sequence of planned vehicles
 - Approach uses start times from roster (false) or own start times (true) with `roster.fifo_does_the_roster`
 - Switch for role of roster group and number is DG-parameter `theContinuousRosterFlag`
- Preplanning provides assigned sign on (SAS) and considers the Roster
- Realtime-Optimization:
 - Planning of vehicles with SAS
 - No Proposals until vehicle is signed on
 - Late sign on, i.e., after SAS, results in preference of late vehicle
 - Early sign on, i.e., prior to SAS, does not result in preference → Exception: Delay can be avoided!

Vehicle Priorities



- Vehicles with higher priority 1 are preferred over vehicles with priority 2, if
 - the delay by using a higher prioritized vehicle is not exceeded over that of a lower prioritized,
 - the MR is not violated, i.e., prioritizing of vehicles does not work for first delivery,
 - a higher prioritized vehicle arrives until the expected load time of a lower prioritized vehicle plus the difference between the DG-parameters `theTruckHoldingTimePrio1` and `theTruckHoldingTimePrio2`
- Vehicle priorities can be configured by `theHaulierCostPriority` at the resource

Early Loading

- Plant capacity = Number of deliveries per hour or quantity per time unit
- Overlap of two deliveries d_1, d_2 :
 - (1) Delay d_2
 - (2) Prefer $d_1 \rightarrow$ „Early loading“
- Optimization tries (2) first
- Maximum early loading can be configured by DG-parameter `theDefaultShelfLife`, i.e., the earliest start loading timestamp is the left side of the delivery time interval minus the shelf time
- Restrictions:
 - Arrival of proposed vehicle
 - Start time of optimization
 - Fixed deliveries
 - Time

Properties and Plant Dispatch

- Properties and exclusions work for the following combinations:
 - Order – shift
 - Unload location – shift
 - Load location – shift → can be configured by `optim.fifoCheckPlantExclusion`
- Plant Dispatch: Allows for vehicles working from home plant
- Deliveries are scheduled at principal plant if available
- If principal plant is not available for loading, e.g., by opening hours, the nearest alternative is used
“Only principal plant usage” can be configured by `optim.fifoAllowInitialMultiplanting`

TRANSFORMATIONS



Approach

- (Optional) second phase (actual optimization) enabled by DG-parameter `theEnableTrafos`
- Work (deliveries) and resources (vehicles) are exchanged between plants
- Objective: Reduce number of remaining and late deliveries, ...
... but avoid excessive travel time
- Approach (iterative):
 - Plant Dispatch for each plant
 - Calculation of (lateness-)costs for each plant
 - Determination of Pre-Selects, i.e., pairs of plants (acceptor, provider) for exchange
 - Generation of predefined number of transformations (selects)
 - Evaluation of transformations
 - Execution of *best* transformation→ Assessment by costs

Assessment of Transformations

- Assessment of overall situation by one transformation t :
 - ➔ Difference of cost at Provider + Difference of cost at Acceptor + Cost of t (additional travel)
- Select transformation with highest improvement of total cost
- Repeat until (1) no more transformation with improvement can be found, i.e., all t with cost ≥ 0 , or (2) the maximum number of transformations is done
- *Iterative approach will be described in more detail in the next section*

Costs

Plant costs consist of four terms

- Delay! (delivery)
- Early Loading (delivery)
- Overtime (vehicle)
- Idle-Times (vehicle)
- DG-parameters:

<code>theScaleEarlyLoad</code>	Weight for early loads
<code>theScaleIdle</code>	Weight for idle times at plant
<code>theScaleOvertime</code>	Weight for overtime at plant
<code>theCapIdle</code>	Minimum for idle time at plant
<code>theCapOvertime</code>	Minimum for overtime at plant

Delay Costs

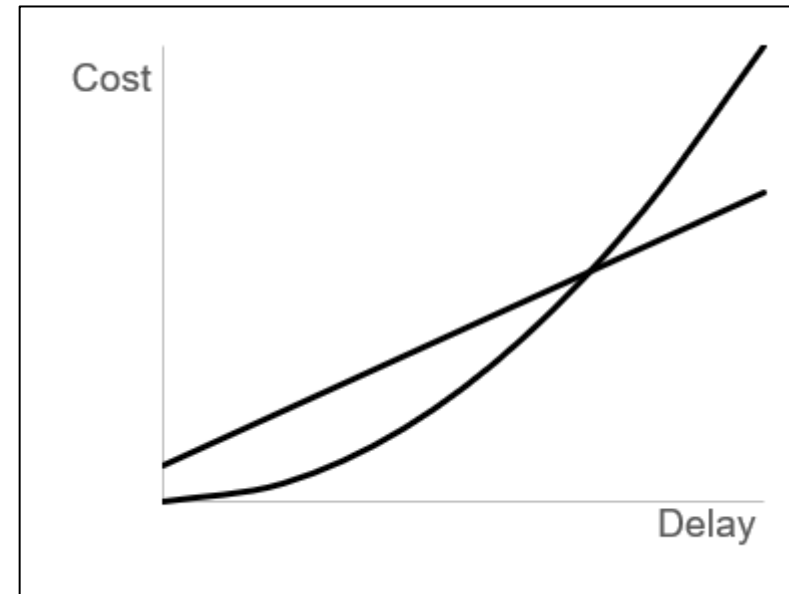
- Delay: Difference between proposed load time and end of load time-interval
- Quadratic and linear parts of cost function with d : delay:

$$\min(m \cdot d + n, c \cdot d^2)$$

- DG-parameters:

m	n
theScaleDelay	theScaleDelayOffset

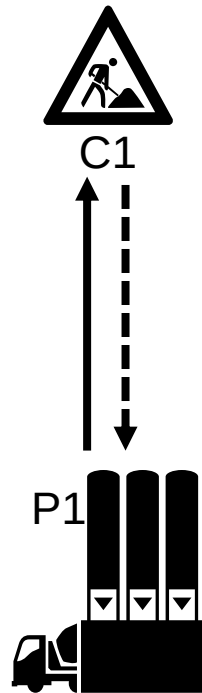
Priority of order	c
0, 1, 5, 6	theScaleSquare1
2	theScaleSquare2
3	theScaleSquare3



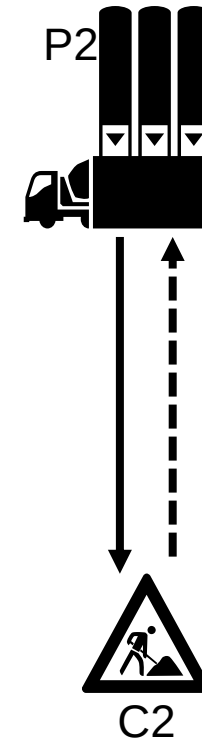
- Plusloads have priority 0 while the default for all orders is priority 2

Types of Transformation

- Plant Dispatch

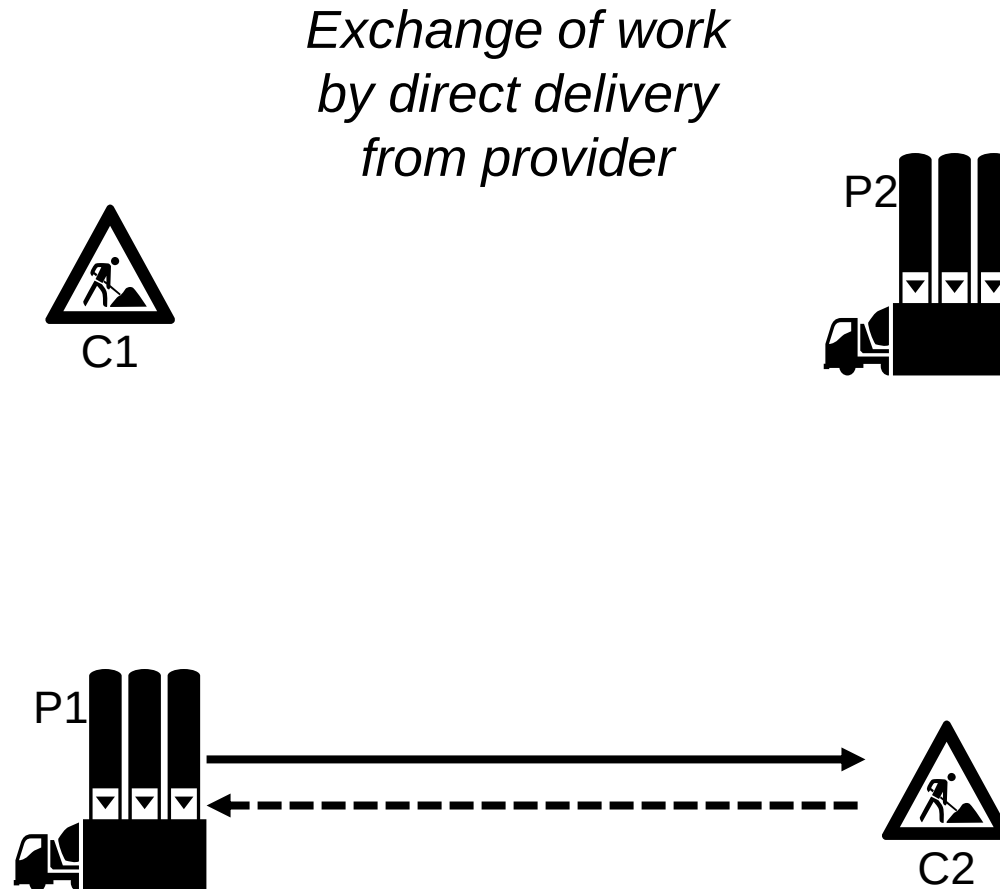


*Delivery from
principal or
closest available
plant*



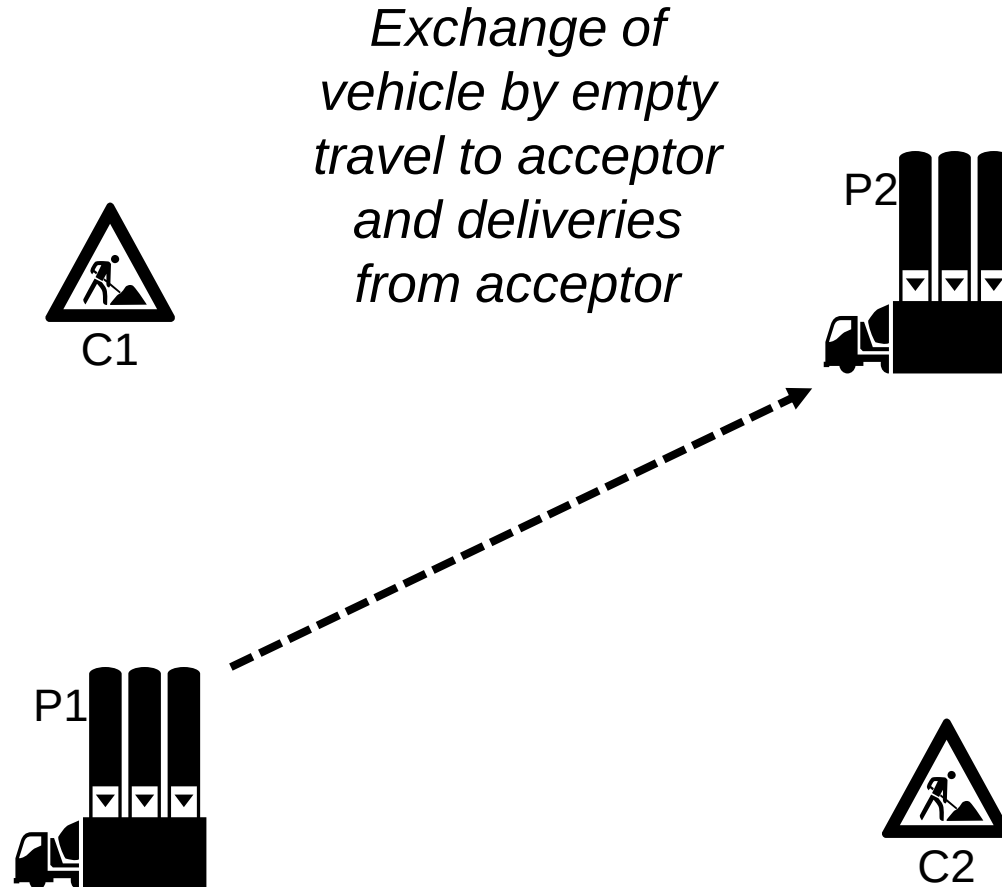
Types of Transformation

- Plant Dispatch
- Four Transformations:
 - Alternative Plant



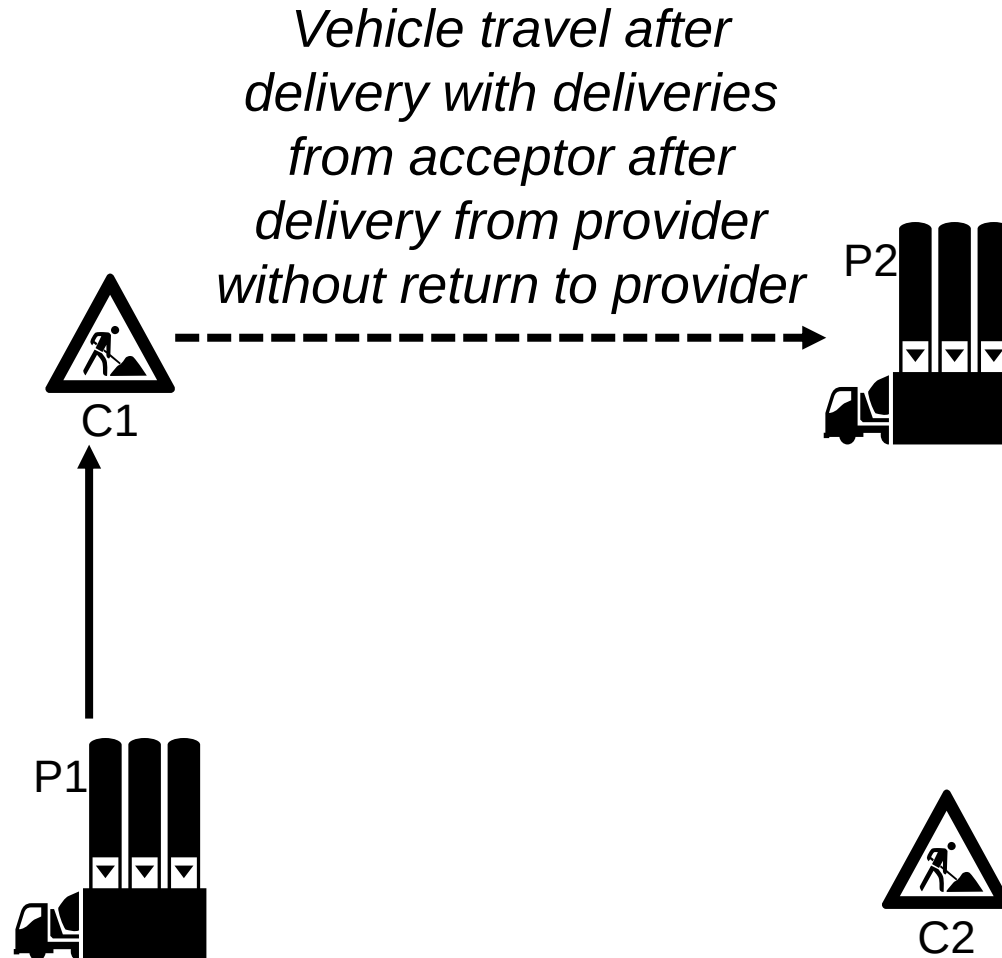
Types of Transformation

- Plant Dispatch
- Four Transformations:
 - Alternative Plant
 - Approach from Plant



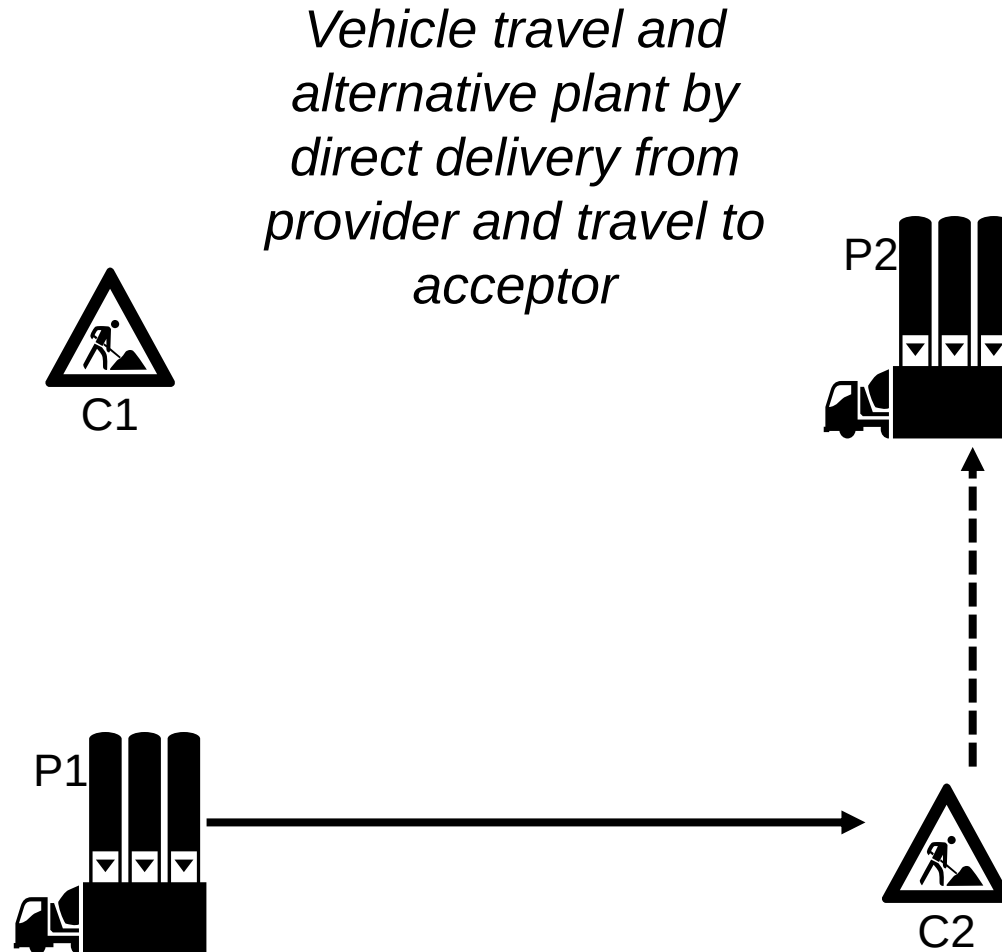
Types of Transformation

- Plant Dispatch
- Four Transformations:
 - Alternative Plant
 - Approach from Plant
 - Approach from Customer



Types of Transformation

- Plant Dispatch
- Four Transformations:
 - Alternative Plant
 - Approach from Plant
 - Approach from Customer
 - Approach by Delivery



Types of Transformation

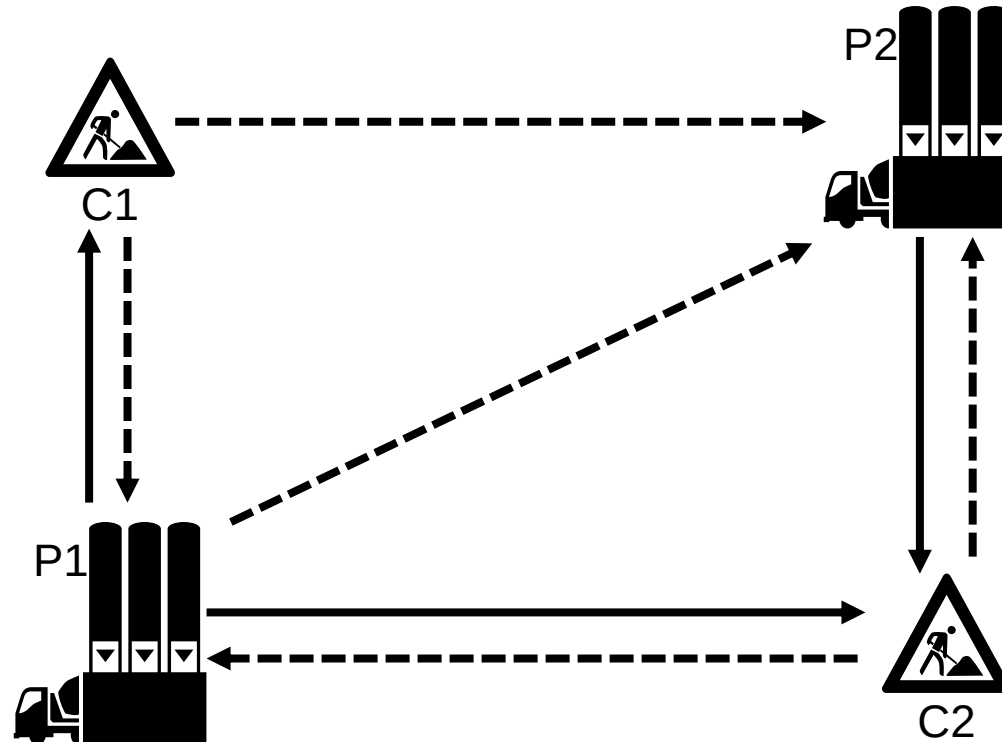
- Plant Dispatch

- Four Transformations:

- Alternative Plant (AP)
- Approach from Plant (AfP)
- Approach from Customer (AfC)
- Approach by Delivery (AbD)

Exchange
resources (vehicles)
between plants

Exchange work
(deliveries)
between plants



Cost Weighting

- Cost of a transformation t can be weighted by configuration:
Difference of cost at Provider + Difference of cost at Acceptor + **Cost of t** (additional travel)
→ Cost must be negative for improvement by transformation t
- DG-parameters to weight **cost of transformation t** :

<code>theScaleAlternativePlant p</code>	Weight for AP-transformations: $2p \cdot \text{additional travel by } t$
<code>theReturnHomeWeight r</code>	Weight for additional distance to home location of vehicle
<code>theScaleApproachFromCustomer p</code>	Weight for AfC- and AbD-transformations $p \cdot (\text{additional return to plant travel by } t + r \cdot \text{additional return to home travel by } t)$ [AfC] $p \cdot (\text{additional approach to unload location travel by } t + r \cdot \text{additional return to home travel by } t)$ [AbD]
<code>theScaleApproachFromPlant p</code>	Weight for AfP-transformations $p \cdot (\text{additional return to plant travel by } t + r \cdot \text{additional return to home travel by } t)$

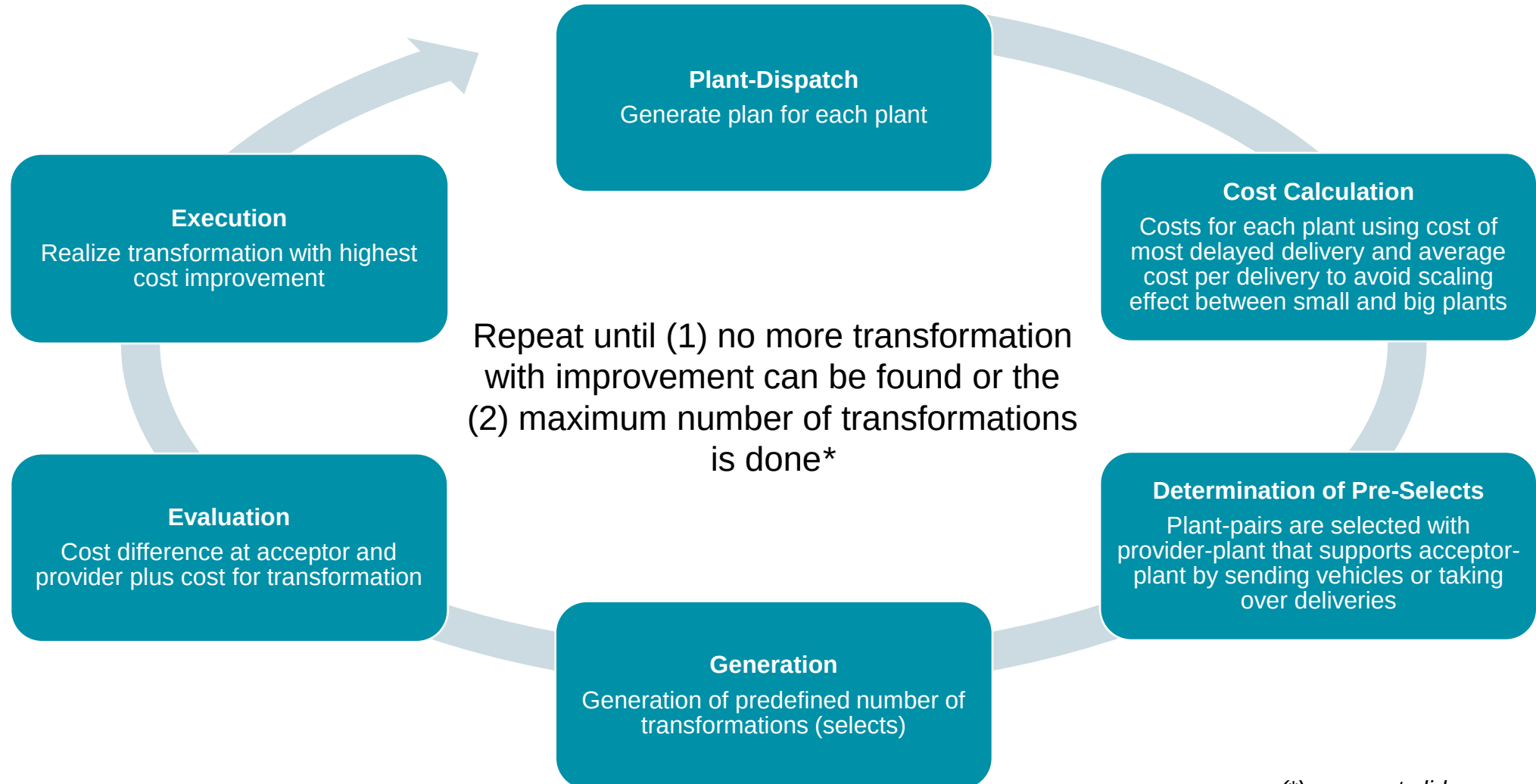
Parameters

DG	<code>thePlantDispatchOnly</code>	Forbid all transformations where plants are changed (AP, AbD)
optim	<code>fifoForbidSomeTrafos</code>	Comma-separated list with string entries AbD, AfC, AfP, AP (see types of transformations)

OPTIMIZATION APPROACH



Iterative Optimization Approach



(*) see next slide

Number of Transformations

(1) No more transformation with improvement can be found, i.e., cost of all transformations is positive

- Force to make at least one transformation even it does not improve by `optim.fifoAtLeastOneTrafo`

(2) Maximum number of transformations is done

- DG-parameter `theMaxTrafosPer100Deliveries` results in maximum of 20 or

$$\frac{p \cdot n}{100}, \text{ n: number of deliveries at plant,}$$

but overwritten by `fear.MaxNofFifoTransformations`, if > 0

Pre-Selects

Calculate cost for each pair of plants by

$$\begin{aligned} & \textbf{Cost of most expensive delivery at provider} \\ & + \textbf{Travel between plants} \\ & - \textbf{Cost of most expensive delivery at acceptor} \end{aligned}$$

Weight of travel duration between plants (provider, acceptor) in case of transformation can be configured by DG-parameter `thePreSelectTruckDurationFactor`

Select best pairs → Find pairs where acceptor has high cost and provider can support = Pre-Selects
DG-parameter for number of selected Pre-Selects: `thePreSelectsPerIteration`

Determine specific transformations for each Pre-Select, i.e., pre-defined number n of transformations per Pre-Select = Selects

- Parameter for number of Selects n per Pre-Select: `theTransformationsPerPreSelect`

Delivery Selects

- $2/3 n$ delivery-Selects (AP)
 - Check each delivery at acceptor if it can be moved to provider, i.a., provider is AP
 - Maximum distance for transformation defined by DG-parameter `theMaxTransferLimit`
 - Maximum distance to return location defined by `optim.fifoMaxReturnDistance`
 - Estimate cost/effort of each moved delivery
 - Delay, time since start of optimization, rare properties, high priorities
 - No exchange of deliveries of First Round without delay
 - Select best $2/3 n$ transformations

Vehicle Selects

- n vehicle-Selects (AbD, AfC, AfP)
 - Check each delivery at acceptor if it can be moved to provider (see previous slide)
 - Determine responsible vehicle at provider for each potentially moved delivery according FiFo-rule
 - Check all vehicles of provider at acceptor and if it is useful for each transformation AbD, AfP, AfC
 - (i) Empty travel, continuation after delivery of (ii) the most suitable own delivery and (iii) the best AP delivery that was determined before
 - Estimate effort for each transformation and select best
 - Availability at acceptor plant – the earlier the better
 - Malus in case of Frist Round-delivery
 - Prefer transformation with each truck type by increasing `fear.fifo_truckSelect_firstOfTypeBonus`
 - Bonus if additional load is possible with `fear.fifo_truckSelect_loadDoneBonus`
 - Select best n transformations

ADDITIONAL PARAMETERS



Additional Parameters

optim	<code>createFullNumberOfAPTrafos</code>	Configure if only two or all (=theTransformationsPerPreSelect) AP-transformations are selected
fear	<code>shelfLifeViolations_areUndeliverable</code>	No proposals, if shelf life is exceeded, e.g., by travel duration
DG	<code>thePlusLoadPercentage</code>	Percentage of average delivery quantity used for Plusload → Small Plusloads potentially delivered with last regular load if vehicle capacity sufficient

CONCLUSION





Overview

- Name: FiFo or Transfer-Optimization
- FiFo-rule, i.e., first vehicle gets next load
- Orders are broken down into deliveries according to the defined load-spacing
- Load-spacing and sequence of deliveries are considered
- Two-step approach with Plant Dispatch (starting point) and Transformations (optional)
- Various constraints like First Round-rule or Morning Roster
- Planning from left to right

Nice to know:

- Only signed on vehicles or vehicles with a SAS are considered, while FAS and fixed deliveries are not considered (except configured by `global.fifoStartPlanningAfterFixedProposals`)
- No multi-shift functionality and planning horizon is one day with hard break at day-change
- Costs are configurable (see parameters)
- Debug-output can be switched on: `msconfig -v"fear.log_optimizer_enable=1"`

DISCUSSIONS AND QUESTIONS

